ASSIGNMENT 4

TOPIC: SOFTWARE TESTING, QUALITY ASSURANCE, METRICS & CONFIGURATION MANAGEMENT

1] Define Software Testing?

Software Testing is the systematic engineering process of executing a software application under controlled conditions with the deliberate intent of identifying defects, errors, or gaps, and verifying that the delivered system conforms to all specified functional and non-functional requirements. Testing validates system behavior against expected outputs, measures performance benchmarks, verifies security defenses, and establishes high confidence in system reliability before production release. In the College AI Chatbot project, testing verified that all 14 FAQ domains returned accurate responses, fuzzy typo algorithms scored correctly, and authenticated sessions remained protected.

2] What is Risk Identification?

Risk Identification is the proactive, continuous project management process of recognizing, categorizing, and documenting potential technical, operational, schedule, or financial vulnerabilities before they materialize into active failures. In our project lifecycle, key identified risks included:

• *Hosting Latency Risk:* Cloud instances sleeping on Render's free tier causing ~30s response delays (mitigated by documenting cold-start behavior and adding responsive loading indicators).

• *Outdated Information Risk:* Stale FAQ answers due to institutional circular revisions (mitigated by engineering an admin portal with dynamic CRUD interfaces).

• *Typographical Ambiguity Risk:* Unusual query phrasing causing matching failures (mitigated by deploying Python difflib similarity scoring).

3] What is SCM?

Software Configuration Management (SCM) is an umbrella software engineering discipline that manages and tracks the evolution of software deliverables throughout the project lifecycle. SCM ensures product baseline integrity by governing four key pillars: Configuration Identification (labeling code, schemas, and documents), Version Control (recording chronological diffs), Change Control (evaluating and approving modifications), and Configuration Auditing. In our project, SCM was enforced using **Git** for version control, **GitHub** for remote repository tracking, and automated deployment hooks for Render.

4] Define Debugging.

Debugging is the diagnostic and corrective engineering activity conducted by developers to locate, analyze, trace, and rectify the underlying root cause of an identified software bug or runtime failure. Unlike testing (which merely demonstrates the existence of an error), debugging investigates internal program states, stack traces, variable values, and control flows to eliminate the flaw. For example, during chatbot development, debugging resolved a SQL datatype mismatch where attendance percentages stored as text strings caused numerical comparison errors in the detention engine.

5] Explain Configuration Audit.

A **Configuration Audit** is an independent formal assessment conducted to verify that a software configuration item conforms to its technical documentation and satisfies all baseline requirements:

• **Functional Configuration Audit (FCA):** Evaluates whether the developed software has successfully satisfied all functional, performance, and operational capabilities specified in the Software Requirements Specification (SRS).

• **Physical Configuration Audit (PCA):** Verifies that the physical build items — source code files, database DDL schemas, configuration manifests, test suites, and user documentation — are complete, correctly labeled, and properly archived in the repository.

6] Differentiate Between Black Box Testing And White Box Testing?

Criteria	Black Box Testing	White Box Testing
Core Focus	Verifies external functional behavior against requirements without knowledge of internal code.	Examines internal logic, source code structure, conditional branches, loops, and data paths.
Code Visibility	Code is treated as an opaque “black box”; inner implementation is completely hidden.	Full structural visibility into source code, algorithms, and database queries.
Performed By	Independent QA testers, end-users, and academic evaluators.	Software developers, backend architects, and unit test engineers.
Techniques	Equivalence Partitioning, Boundary Value Analysis, Decision Table Testing, State Transition.	Statement Coverage, Branch/Decision Coverage, Path Coverage, Condition Testing.
Project Example	Entering “When will Wagholi admissions close?” and verifying that the correct admission deadline appears in the chat UI.	Executing test_suite.py to test internal loop scoring in match_query() and PBKDF2 salt validation in check_password_hash().

7] What Do You Mean By Metrics?

Software Metrics are standardized, quantitative measurements used in software engineering to assess, track, and benchmark the size, quality, performance, complexity, and productivity of software products and development processes:

• *Product Metrics:* Lines of Source Code (SLOC), Cyclomatic Complexity, Database Table Count (4 normalized tables), and API Response Latency (< 250 milliseconds).

• *Process Metrics:* Defect density, test execution pass rate (100% pass on test_suite.py), and build failure frequency.

• *Chatbot Operational KPI Metrics:* Our admin command center displays live metrics: **Total Active FAQs**, **Total Queries Answered**, **Active Students**, and **Satisfaction Percentage** calculated from thumbs up/down user feedback.

8] What Do You Mean By Version Control?

Version Control (also known as Source Code Management or Revision Control) is an automated system that records changes to a set of files over time, enabling developers to recall specific historical revisions, compare chronological diffs, branch experimental features, and merge concurrent contributions without overwriting code. Our project utilized **Git** for local branch management and **GitHub** as the cloud repository. Git commit hashes provide complete cryptographic traceability for every bugfix, while Render's CI/CD pipeline listens to GitHub push events to deploy verified code to production automatically.

9] Explain Object Oriented Software Engineering?

Object-Oriented Software Engineering (OOSE) is a comprehensive software engineering paradigm that models a software system as a collaborative community of interacting “objects” that encapsulate both state (attributes) and behavior (methods), mirroring real-world domain entities. OOSE builds upon core principles:

• **Abstraction:** Exposing essential features while suppressing unnecessary low-level implementation details.

• **Encapsulation:** Bundling data and methods into isolated classes (e.g., Student class guarding roll number and attendance fields).

• **Inheritance:** Reusing common attributes through hierarchical generalization (e.g., Student and Admin inheriting from User).

• **Polymorphism:** Enabling different classes to respond to standard method calls in class-specific ways.

OOSE was applied in our project by translating UML Class and Use Case specifications directly into modular Flask blueprints and normalized relational database tables.

10] Explain Various Types of Testing in details.

1. **Unit Testing:** Testing the smallest testable units of code (individual functions or classes) in strict isolation. Application: Testing the difflib similarity scoring logic inside match_query() with synthetic inputs.
2. **Integration Testing:** Verifying that distinct software modules interface and exchange data correctly when combined. Application: Validating that the Flask /chat route correctly queries SQLite3, inserts history records, and returns compliant JSON.
3. **System Testing:** Evaluating the completely integrated software application as a unified system to verify end-to-end specifications. Application: Testing the complete web interface on Render across desktop and mobile devices.
4. **Acceptance Testing (UAT):** Validating that the system satisfies business goals and user expectations. Application: Demonstrating to project guide **Asst. Prof. Sheela Satav** that students can log in, check attendance percentages, and receive voice alerts.
5. **Regression Testing:** Re-executing existing test suites whenever source code is altered to confirm that bugfixes have not introduced secondary defects. Application: Re-testing FAQ matching after implementing the Dark/Light UI theme toggle.
6. **Performance & Load Testing:** Benchmarking response times and throughput under peak concurrent user loads. Application: Verifying sub-250ms API query latency during simulated simultaneous student queries.
7. **Security Testing:** Inspecting software for vulnerabilities. Application: Testing for SQL injection using parameterized queries, verifying salted PBKDF2 password hashes, and preventing Cross-Site Scripting (XSS).
8. **Usability & Accessibility Testing:** Assessing ease of use and accessibility. Application: Verifying Web Speech API audio clarity and mobile tap target sizes.

11] Differentiate Between automated Testing And Manual Testing.

Comparison Factor	Manual Testing	Automated Testing
Execution Method	Executed manually by a human tester interacting step-by-step with the user interface.	Executed automatically by testing scripts, frameworks, and continuous integration runners.
Execution Speed	Slow, laborious, and limited by human typing and visual review speeds.	Extremely fast; hundreds of test assertions execute in fractions of a second.
Human Resource & Fatigue	Prone to human error, boredom, and missed defects during repetitive regression cycles.	100% consistent, tire-free, and repeatable with identical precision every execution.
Initial Cost vs. ROI	Low initial setup cost; however, ongoing labor costs remain high over time.	Higher initial development cost to write test scripts; exceptionally high long-term ROI.
Best Suited For	Exploratory testing, UI/UX aesthetics, usability evaluations, and ad-hoc visual checks.	Regression testing, unit test suites, load/performance testing, and CI/CD validation.
Application in College AI Chatbot	Used to verify speech synthesis clarity, microphone voice recognition in ambient campus environments, dark mode visual contrast, mobile touch responsiveness, and admin dashboard usability across Chrome, Safari, and Edge.	Implemented in test_suite.py using Python's unittest framework. Automatically asserts: • Correct FAQ response matching for all 14 categories. • Typo-tolerant threshold validation (“hostal” → Hostel). • Student attendance lookup and 4-tier detention status math. • Admin password hashing cryptographic verification (100% pass rate).

Quality Assurance & Testing Conclusion

By deploying a hybrid testing strategy combining automated unit test suites with rigorous manual cross-browser usability evaluations, the **College AI Chatbot** achieved a verified defect-free baseline. The system delivers sub-250ms query latency, handles spelling variations with fuzzy scoring, protects administrative routes with salted cryptographic hashes, and ensures seamless academic self-service for all students of **G H Raisoni International Skill Tech University, Pune**.